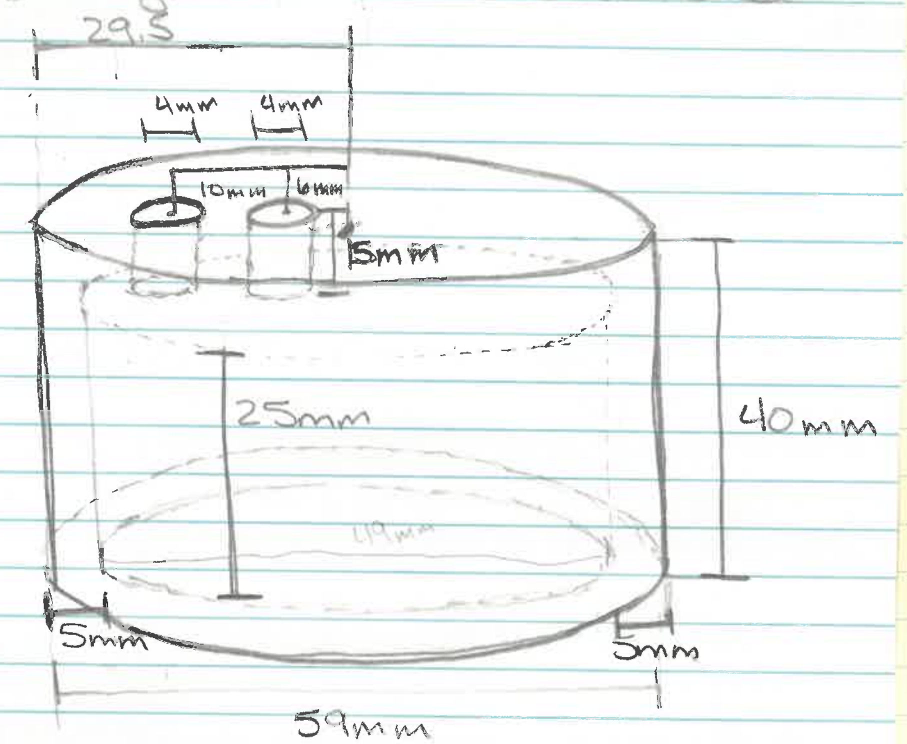
# Continuous Water-level Sensor Based on Load Cell and Floating Pipe

## Parts List:

- 1kg Load Cell x2
- HX 711 sensor x2
- Float Load Cell Holders x2
- Load Cell Holder Mount x2

Design Float Load Cell Holder

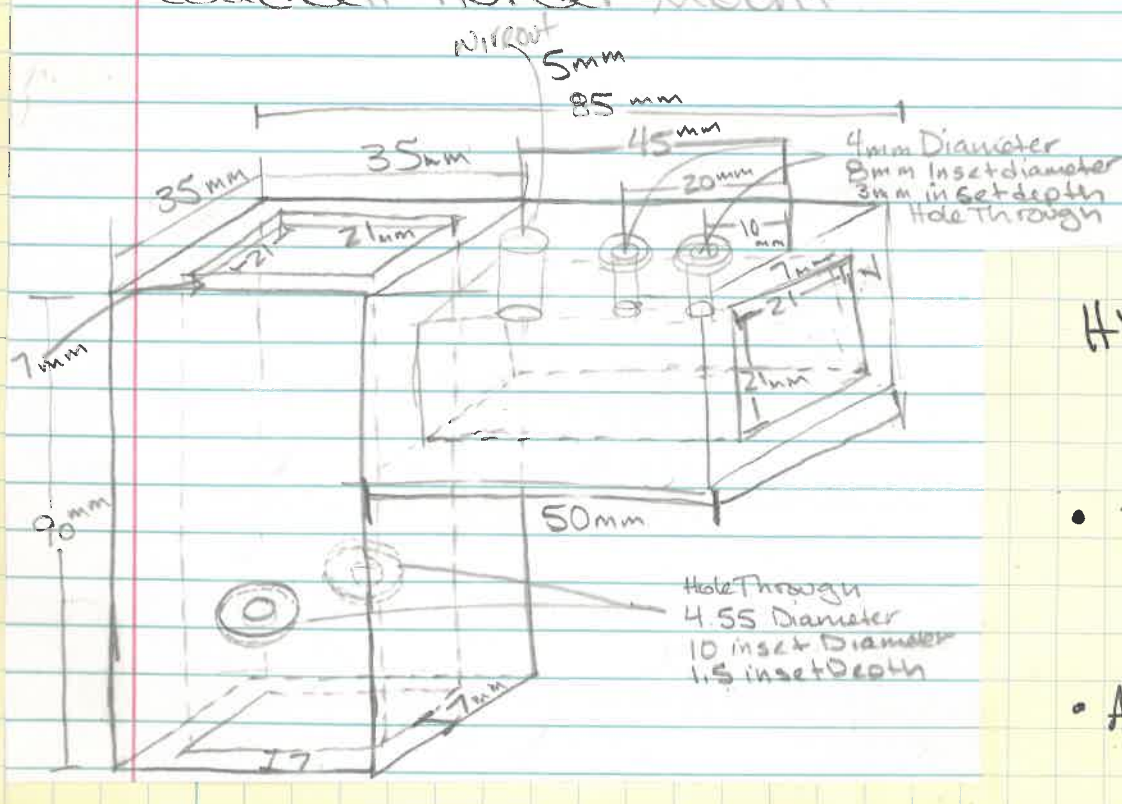



Cad design
3-D Printed
M4-.70 x ~~35~~ mm Screws too Long
to attach LoadCell

- 2 Ft Long 1.5" PVC Pipe
- Aluminum Square (20mm) Extruded Rod apox 2FT
- Extruded Aluminum Mounting Hardware
- Mounting Base Support (Design Next Page)
- Self Tapping #10 screws (For Base)
- Bottom Cap Flex Sealed
- JB Weld Plastic Bond to attach Caps

Cad design
3-D Printed
M4-.70 x 20mm Screws
to attach LoadCell

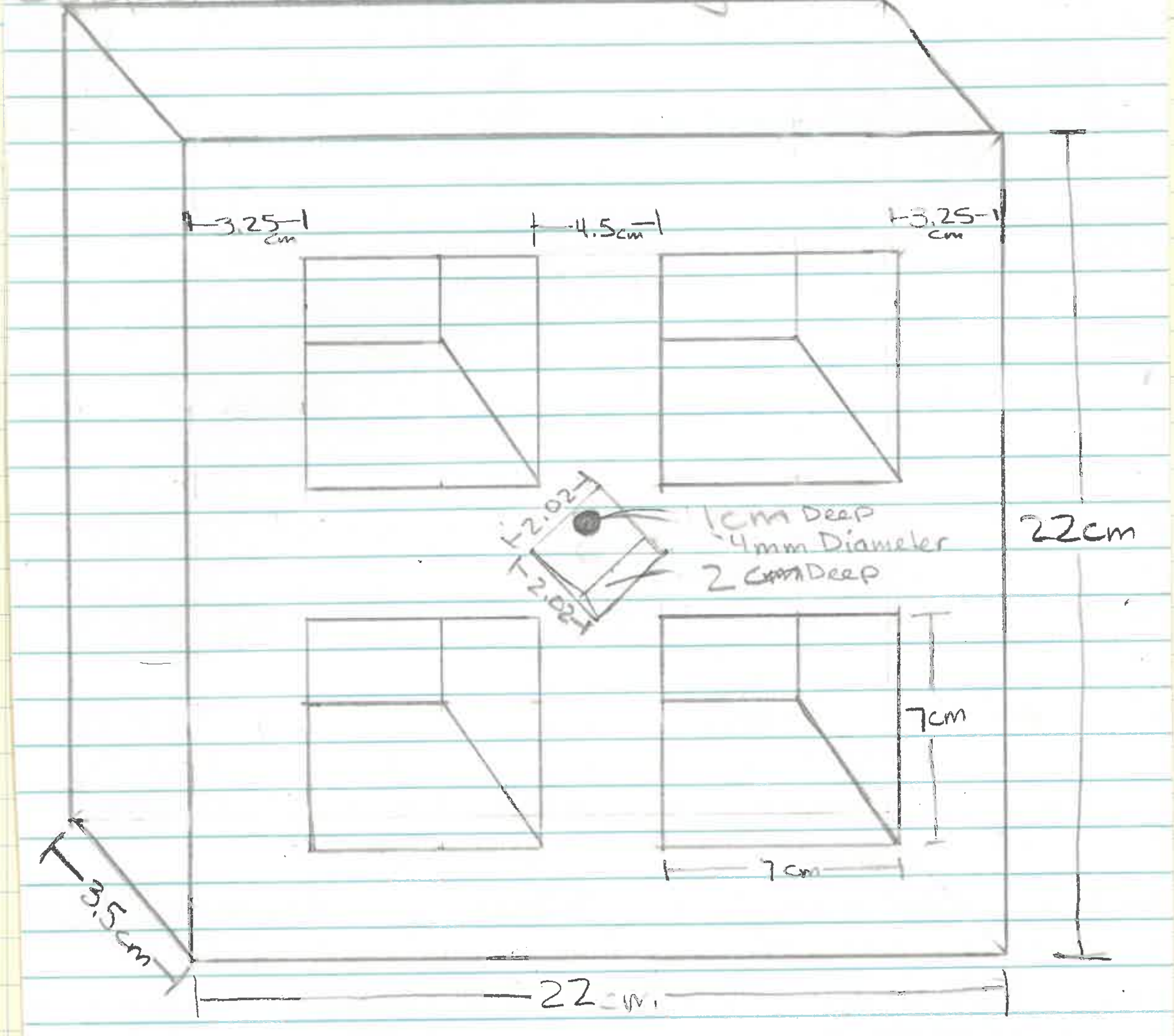
LoadCell Holder Mount




HX711 Connected/Contained Within + Potted

- 22/4 AlarmWire Connecting HX711 to Arduino
- Arduino

## Support Base For Mounting




Cad Design
3-D Printed

cutouts on Each Cross Bar For adding Weight
2 x 7cm x 2.5cm depth

Bottom Side Hole Through Countersink
diameter 10mm
depth 4mm

## HX711 Wiring Diagram

| Load Cell | HX711 | | | 22/4 Alarm wire To Arduino | | Arduino | |
|---|---|---|---|---|---|---|---|
| Red | E+ | Gnd | Green | | G | Ground | |
| Black | E- | DT | Black | | B | Pin 2 /4 | DT |
| White | A- | SCK | Yellow | | Y | Pin 3 /5 | SCK |
| Green | A+ | +5V | Red | | R | +5V | |

Pins 2 and 3 → Upper Load Cell

Pins 4 and 5 → Lower Load Cell (inverted)

## Archimedes Principle

$F_b = -\rho g V$

$\rho_{water} = 1 \times 10^{-6}$ Kg/mm3

$g = 9807$ mm/s$^2$

$V = \pi r^2 \cdot d$

$F_b^{Expected} = 6712.988$ Kg·mm/s$^2$

$= 6.713$ N

$\approx 684.54$g

Calculating Expected Value of about ½ way UpTube Submerged

Tube $(\pi \cdot (24.5\text{mm})^2 \cdot 305\text{mm})$
$= 575151.002$ mm$^3$

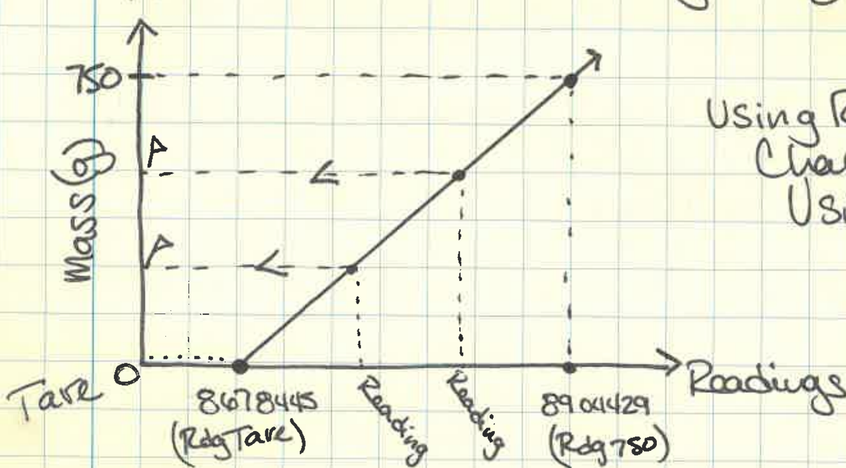
Cap $(\pi \cdot (29.5\text{mm})^2 \cdot 40\text{mm})$
$= 109358.8403$ mm$^3$

Total V $= 684509.8423$ mm$^3$

## Calibration

| | Upper | Lower |
|---|---|---|
| Tare | 8678445 | 8605090 |
| 750g | 8904429 | 8262529 |

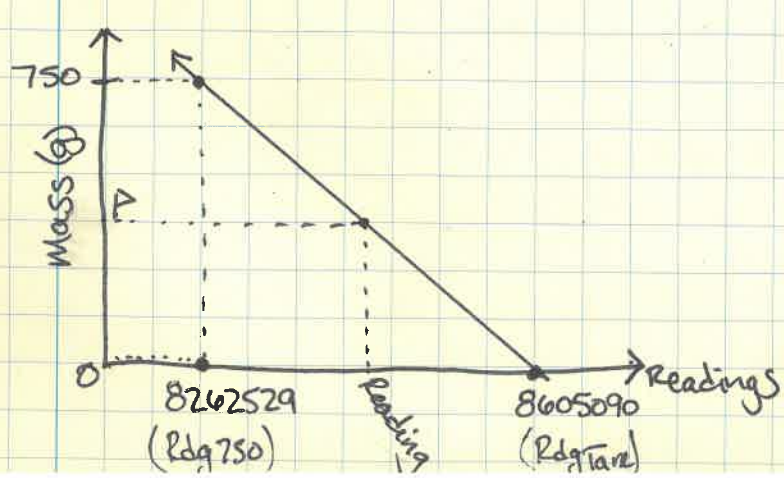
Upper




Using Readings can determine Change in masses.
Using Calculations

$$M_u = \frac{750.0\text{g}}{Rdg_{750} - Rdg_{Tare\,u}} \left(\text{Current Reading}_{Upper} - Rdg_{Tare\,U}\right)$$

Lower (inverted)




Calculation For m will be the Same and the negatives will cancel out.

$$M_L = \frac{750.0\text{g}}{Rdg_{750} - Rdg_{Tare\,L}} \left(\text{Current Reading}_{Lower} - Rdg_{Tare\,L}\right)$$

Finding Force Using Both Upper + Lower LoadCell Readings

$$(M_U + M_L) \times 9.8 / 2 = F_b$$

$F_b = \rho g(A \times d)$ = To Solve for Finding the distance of water level Rise / Fall (d) Use $F_b$ calculated and solve for (d) the distance. Because density of water ($\rho$) and gravity (g) cannot be changed Accuracy can only be more accurate by changing Area calculation making the distance flucuation more precise

$F_b = (10^{-6} kg/mm^3)(9800 mm/s^2)\pi r^2 d$

$$d = \frac{F_b}{(10^{-6} kg/mm^3)(9800 mm/s^2)(\pi)(r^2)}$$

Depth = Force / calibration Constant For depth

The issue with getting an Exact depth Calibration Constant has to do with the Fact that a small Portion of our Floating Pipe has a different $r^2$ value. This will mean that the cap with $r = 29.5$ mm and the pipe with 24.5mm = r will create variation in where the calibration is most accurate / precise. An appropriate adjustment must be made and tested to verify best fit.

- Using a value closer to r = 24.5 will ensure that water level changes at higher depths will be more precisely measured
- Using a value closer to r = 29.5 will ensure water level changes closer to the cap will be more precisely measured.

* ~~Assuming a Force for Example~~ ~~$F_b = 5.8$~~

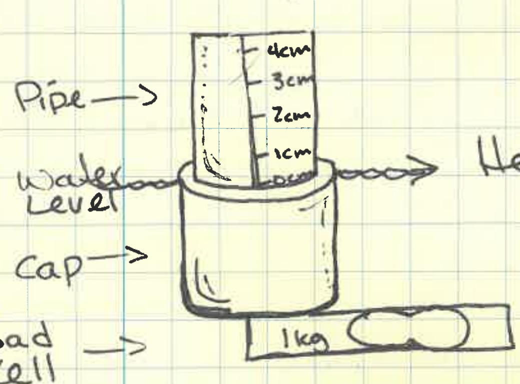
$$d^{Pipe} = \frac{F_b}{(10^{-6} kg/mm^3)(9800 mm/s^2)(\pi)(24.5 mm)^2} = F_b / 18.48$$

* Large Variation

$$d^{Cap} = \frac{F_b}{(10^{-6} kg/mm^3)(9800 mm/s^2)(\pi)(29.5 mm)^2} = F_b / 26.79$$

To combat and correct the Complication with the Calibration the Cap will be zeroed out of the calculation and This Systematic Error will be of no consequence.

* How to Correct for this Systematic Error
- Fill the water to the top edge of cap and then Start the Calibration Procedure The Top of the cap is now the Zero distance starting point.




Here $D_{epth} = 0$ and

Force = 0


## To Run the Buoyance Force Calculation Program

1. Set the Zeroed Out water level (Fill to Top Edge of Cap)
2. Begin the Program and Follow the directions on the Serial Monitor
   - Enter upper and Lower Tare Values
   - Enter Upper and Lower 750g Values
     ↳ Remove 750g weight / Verify Zeroed Value.
3. Now Begin to Raise / Lower Water levels allowing water movement to cease as much as possible and record the Data Values given by the program.

Some Systematic Errors to be Careful of ...
- in the current setup the bottom of the Water Container will flex under the weight of water at some point this can cause

the entire divice to shift and the measured values will have some variation up and down and must be carefully watched for to ensure consistency.

→ This Error can be corrected with a Better watercontainer that sits flat at all times and is confirmed with a level prior to running the Calibration Procedure

- Another Error that must be carefully watched for is water movement. If the water is sloshing in the container and around the device the Tape Measured Distance Value can be slightly off as well as the force values can vary greatly.

→ This Error can be corrected by allowing the water movement to settle down to as near still as is reasonably achievable before taking any measurements

- A Third Error to Consider is the Ability of the Entire device to shift at the base and along the Extruded Aluminum Rod.

→ This will be corrected with a heavier/more permanent base attachment as well as a tighter Fitting Base to Extruded Aluminum Rod attachment.

→ Currently heavy weights are being used but at a point it seems the base is still able to shift and level conditions are difficult to ensure/maintain.

<u>Some other Minor Considerations</u>

- Temperature changes in Water may cause a shrink/ or Expansion of the pipe which can effect the $r^2$ value
- Temperature changes can also effect the shrink/ or expansion of the attached measuring tape
- Placement of the measuring tape or zeroing out position in water can be slightly off.

· measured by tape positions may be slightly off due to water movement or visual discrepancy

## Data Collection

→ ~~Eq~~ Starting from zeroed position increasing water levels by 2 cm steps each time to 32cm
→ at 32 cm level begin to decrease by odd values at 31cm down by 2 cm increments to 1cm level.
→ Can confirm the zeroed out position once movement and shifting errors have been corrected. (This has not happened as of the following data collection it will be addressed for next steps. 0 was only checked at start)

$H^2$ Float Calibrating Program Used for Initial Data Collection

```
#include<Q2HX711.h>                    //Calls HX711 Library
Q2HX711 UpperHX711(2,3);               //Identifies pins for Upper Hx711
Q2HX711 LowerHX711(4,5);               //Identifies pins for Lower HX711
long UpperTare;
long LowerTare;
long UpperWeightedTare;
long LowerWeightedTare;
void setup()
{
 Serial.begin(9600);
 long SumUpper = 0;
 long SumLower = 0;
 for(int ii = 0; ii < 100; ii++)        //Takes 100 readings of each the Upper Tare and Lower Tare
  {
  long UpperReading = UpperHX711.read();
  long LowerReading = LowerHX711.read();
  SumUpper = SumUpper + UpperReading;
  SumLower = SumLower + LowerReading;
  }
 long AverageUpper = SumUpper/100;      //Averages the Upper Tare readings
 long AverageLower = SumLower/100;      //Averages the Lower Tare readings
 Serial.print("UpperTare ");            //Prints out the Upper Tare Average
 Serial.print(AverageUpper);
 Serial.print("\t");
 Serial.print("LowerTare ");            //Prints out the Lower Tare Average
 Serial.println(AverageLower);
 Serial.println ("Add 750 grams of weight     //Add 750g weight before continuing
 Serial.println("Input UpperTare Value");     //Holds program For Serial Monitor input
```

```
while(Serial.available()==0)
{}
UpperTare = Serial.parseInt();
Serial.print("UpperTare input = ");
Serial.println(UpperTare);
Serial.println("Input LowerTare Value");       //Holds program For Serial Monitor input
while(Serial.available()==0)
{}
LowerTare = Serial.parseInt();
Serial.print("LowerTare input = ");
Serial.println(LowerTare);
long SumWUpper = 0;
long SumWLower = 0;
for(int ii = 0; ii < 100; ii++)        //Takes 100 readings of each the Upper 750g and Lower 750g
{
long UpperReading = UpperHX711.read();
long LowerReading = LowerHX711.read();
SumWUpper = SumWUpper + UpperReading;
SumWLower = SumWLower + LowerReading;
}
long AverageWeightedUpper = SumWUpper/100;     //Averages the Upper 750g readings
long AverageWeightedLower = SumWLower/100;     //Averages the Lower 750g readings
Serial.print("UpperWeightedTare ");            //Prints out the Upper 750g Average
Serial.print(AverageWeightedUpper);
Serial.print("\t");
Serial.print("LowerWeightedTare ");            //Prints out the Lower 750g Average
Serial.println(AverageWeightedLower);
Serial.println("Input UpperWeightedTare Value");  //Holds program For Serial Monitor input
while(Serial.available()==0)
{}
UpperWeightedTare = Serial.parseInt();
Serial.print("UpperWeightedTare input = ");
Serial.println(UpperWeightedTare);
Serial.println("Input LowerWeightedTare Value");  //Holds program For Serial Monitor input
while(Serial.available()==0)
{}
LowerWeightedTare = Serial.parseInt();
Serial.print("LowerWeightedTare input = ");
Serial.println(LowerWeightedTare);
}
```

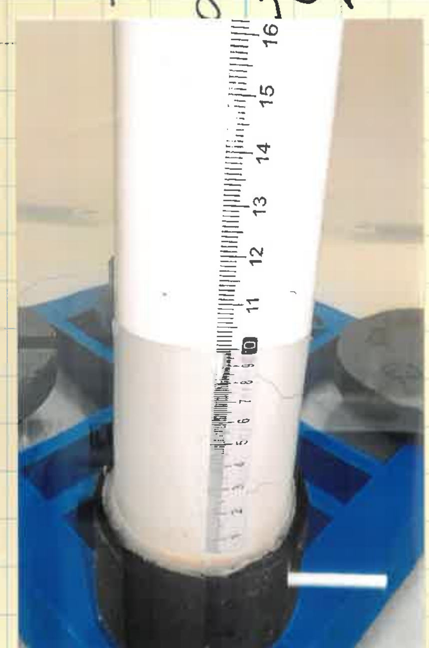
The Void Loop Reads out Each Variable that goes into the Depth Calculation

- I used This information to Ensure all pertinant information was as correct as expected from Other Calculations. It Reads out to 4 decimal places and the Depth is in meters Reads .0125m ⇒ 1.25cm

```
void loop()
{
long UpperReading = UpperHX711.read();          //Each time Upper load cell takes reading
long LowerReading = LowerHX711.read();          //Each time Lower load cell takes reading
float MassUpper = 0.75*(UpperReading-UpperTare)/(UpperWeightedTare-UpperTare);
float MassLower = 0.75*(LowerReading-LowerTare)/(LowerWeightedTare-LowerTare);
float Force = (MassUpper + MassLower)*9.8/2;      //Force Calculation
float Depth = Force/18.5;                          //Distance of water level rise
Serial.println("MassUpper MassLower Force Depth");  //Prints out the float data values
Serial.print(" ");
Serial.print(MassUpper,4);
Serial.print(" ");
Serial.print(MassLower,4);
Serial.print("  ");
Serial.print(Force,4);
Serial.print(" ");
Serial.println(Depth,4);
delay(10000);
}
```

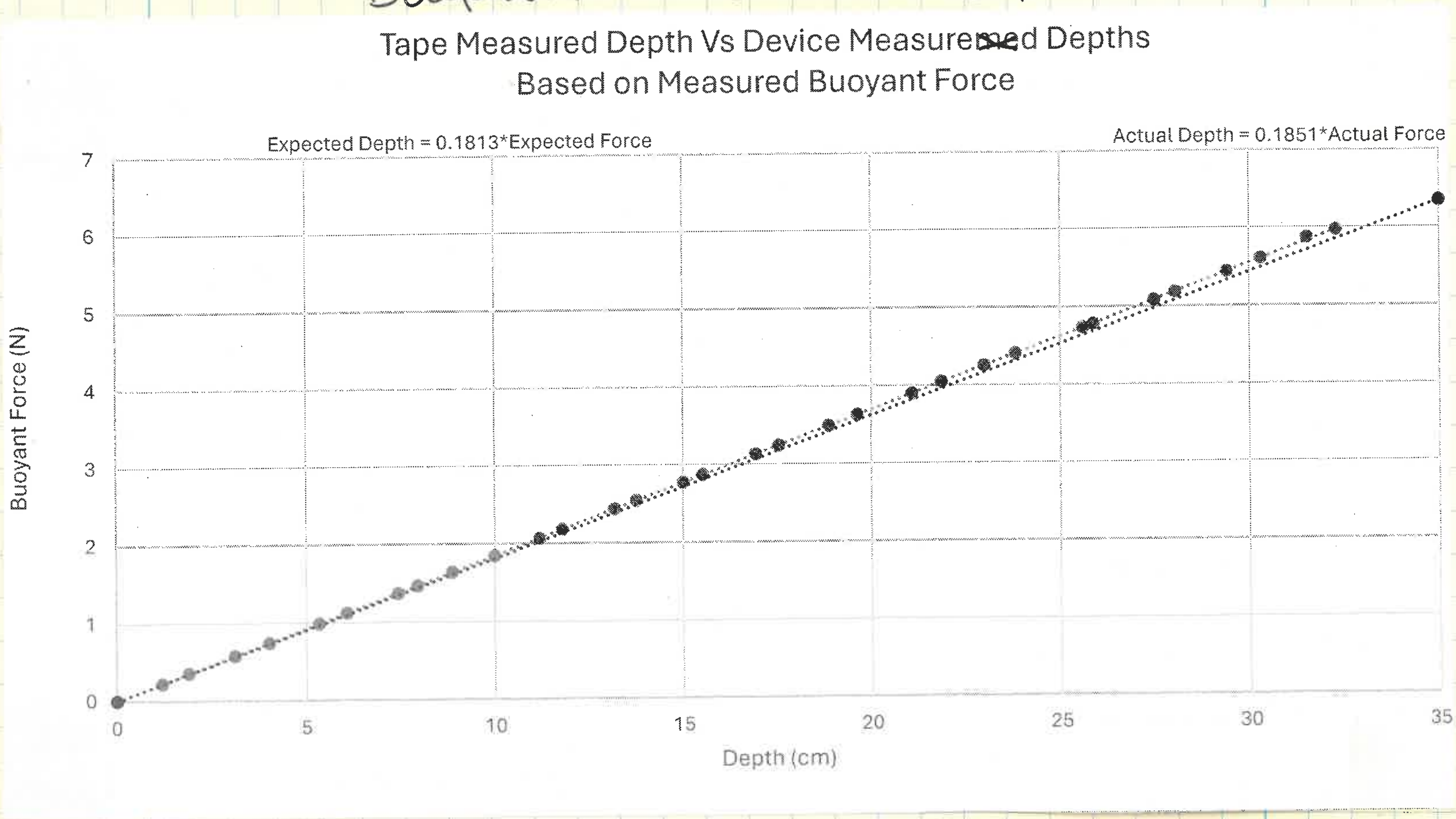
Initial Data Collection

after Calibration Confirmation of the Depth Calculation (Bottom Page) With Pipe Only Zeroed at TopEdge of Cap.

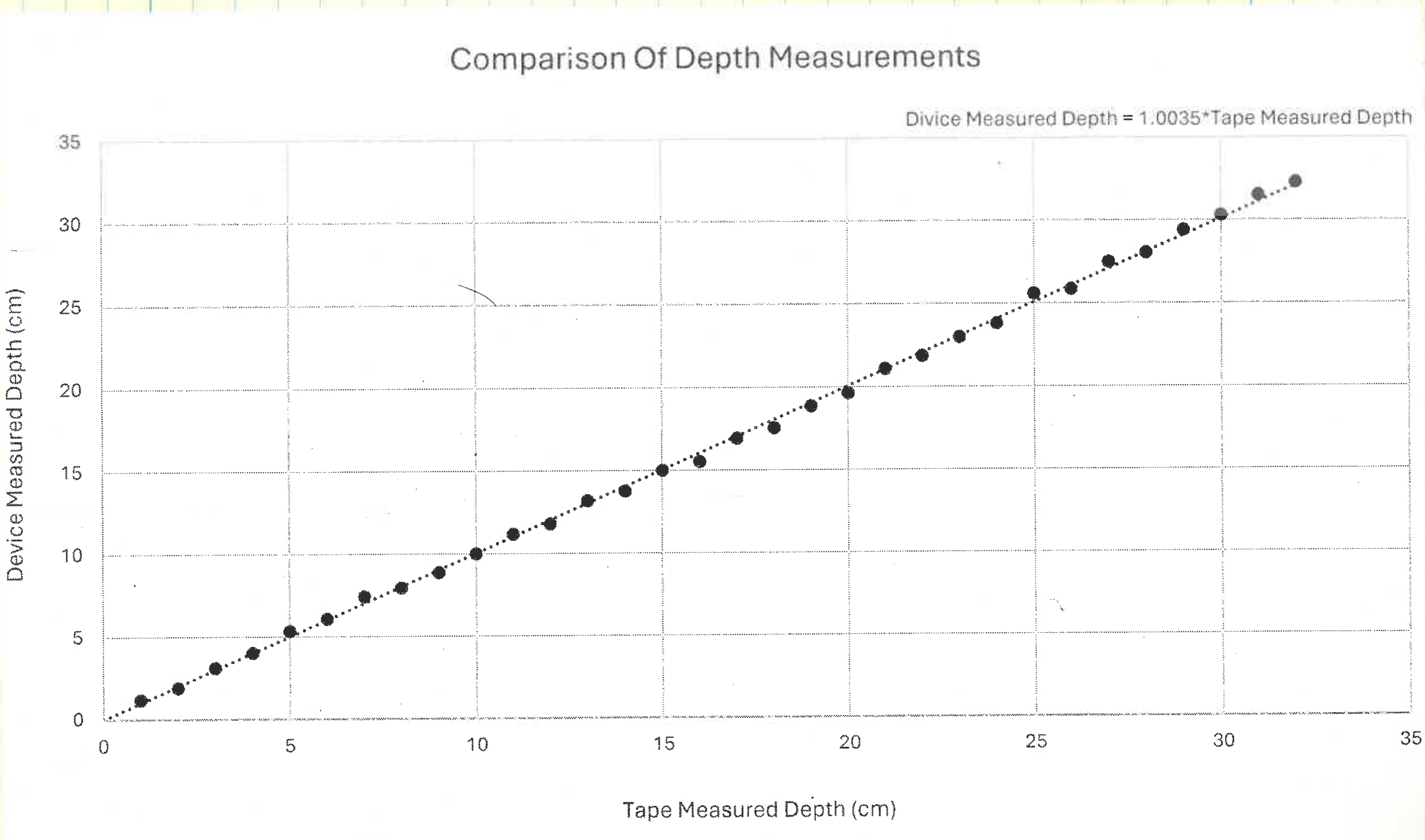


| Tape Measured Depth (cm) | Mass Upper (g) | Mass Lower (g) | Force (N) | Device Measured Depth (cm) | Depth Difference (cm) |
|---|---|---|---|---|---|
| 0 | -0.0042 | 0.0022 | -0.0099 | -0.0005 | 0.00 |
| 1 | 0.0704 | -0.0253 | 0.2208 | 1.19 | -0.19 |
| 2 | 0.0625 | 0.0094 | 0.3523 | 1.90 | 0.10 |
| 3 | 0.0842 | 0.0337 | 0.5775 | 3.12 | -0.12 |
| 4 | 0.1116 | 0.0402 | 0.7437 | 4.02 | -0.02 |
| 5 | 0.1306 | 0.0711 | 0.9883 | 5.34 | -0.34 |
| 6 | 0.1615 | 0.0687 | 1.1236 | 6.07 | -0.07 |
| 7 | 0.1661 | 0.1143 | 1.3740 | 7.43 | -0.43 |
| 8 | 0.1857 | 0.1144 | 1.4702 | 7.95 | 0.05 |
| 9 | 0.1702 | 0.1648 | 1.6414 | 8.87 | 0.13 |
| 10 | 0.2139 | 0.1632 | 1.8479 | 9.99 | 0.01 |
| 11 | 0.2388 | 0.1833 | 2.0683 | 11.18 | -0.18 |
| 12 | 0.2444 | 0.2007 | 2.1809 | 11.79 | 0.21 |
| 13 | 0.2709 | 0.2265 | 2.4377 | 13.18 | -0.18 |
| 14 | 0.2814 | 0.2383 | 2.5461 | 13.76 | 0.24 |
| 15 | 0.2966 | 0.2703 | 2.7775 | 15.01 | -0.01 |
| 16 | 0.3061 | 0.2799 | 2.8716 | 15.52 | 0.48 |
| 17 | 0.3240 | 0.3153 | 3.1329 | 16.93 | 0.07 |
| 18 | 0.3463 | 0.3159 | 3.2446 | 17.54 | 0.46 |
| 19 | 0.3494 | 0.3630 | 3.4911 | 18.87 | 0.13 |
| 20 | 0.3912 | 0.3501 | 3.6323 | 19.63 | 0.37 |
| 21 | 0.4075 | 0.3884 | 3.8999 | 21.08 | -0.08 |
| 22 | 0.4378 | 0.3876 | 4.0442 | 21.86 | 0.14 |
| 23 | 0.4518 | 0.4164 | 4.2539 | 22.99 | 0.01 |
| 24 | 0.478 | 0.4209 | 4.4043 | 23.81 | 0.19 |
| 25 | 0.5446 | 0.4217 | 4.7350 | 25.59 | -0.59 |
| 26 | 0.5305 | 0.4459 | 4.7843 | 25.86 | 0.14 |
| 27 | 0.588 | 0.4494 | 5.0831 | 27.48 | -0.48 |
| 28 | 0.5781 | 0.4803 | 5.1864 | 28.03 | -0.03 |
| 29 | 0.6055 | 0.5045 | 5.4391 | 29.40 | -0.40 |
| 30 | 0.6452 | 0.4988 | 5.6058 | 30.30 | -0.30 |
| 31 | 0.6487 | 0.5410 | 5.8729 | 31.51 | -0.51 |
| 32 | 0.6694 | 0.5492 | 5.9712 | 32.28 | -0.28 |

Buoyant Force vs Water Depth




Device Measured Depth vs Tape Measured Depth



# Setting Up LoRa

MKR1310
VCC → 3.3V

0V – 3.3V Logic

While(!Serial)
Won't Run While Serial Monitor Is Closed

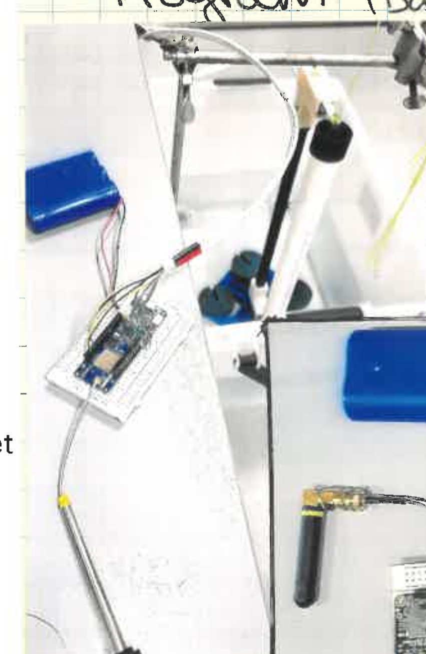
$H_2$ Float Data Header ⇒ H

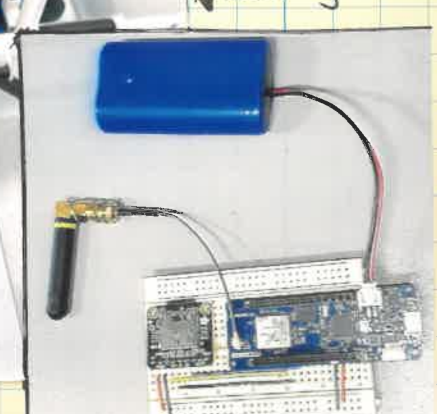
## LoRa Sending Program (Basic)

```
#include <SPI.h>     //Calls SPI Library
#include <LoRa.h>  //Calls LoRa Library

int counter = 0;  //Creates a counter

void setup()
{
Serial.begin(9600);
Serial.println("LoRa Sender");
if(!LoRa.begin(915E6))              //915 MHz
 {
 Serial.println("Starting LoRa failed");
 while(1);   //If LoRa fails, it jams up
 }
}

void loop()
{
Serial.print("Sending packet: ");
Serial.println(counter);            //Shows packet to be sent
LoRa.beginPacket();                 //Opens Packet to Send
LoRa.print("H");                    //Header to identify H2Float Data
LoRa.print(counter);                //Puts data in packet
LoRa.endPacket();                   //Closes packet
counter++;                          //Counter increases by 1 each packet
if(counter > 25)                    //Counter caps at 25
 {
 counter = 0; // 0 to 25 only       //Counter starts at 0 once it hits 25
 }
delay(5000);
}
```

```
#include <SPI.h>     //Calls SPI Library
#include <LoRa.h>    //Calls LoRa Library

void setup()
{
 Serial.begin(9600);
 while(!Serial);
 Serial.println("LoRa Receiver");
 if(!LoRa.begin(915E6))              //915 MHz
  {
  Serial.println("Starting LoRa failed");
  while(1);                    //If LoRa fails, it jams up
  }
}

void loop()
{
 float packetSize = LoRa.parsePacket();     //Get the packet
 if(packetSize)
  {
  char rcvd = LoRa.read();
  if(rcvd == 'H')                //Packet identifier
   {
   while(LoRa.available())
    {
    float data = LoRa.parseFloat();
    Serial.println(data);           //Prints the packet as float
    }
   }
  }
}
```

↑ LoRa Receiving Program (Basic)





Sending|Receiving Data Via LoRa on Battery Power

← Sending

↓ Receiving

Data Taken to Make Pre-Calibration Averages →

| Upper Tare | Lower Tare | Upper 750g | Lower 750g |
|---|---|---|---|
| 8577761 | 8400778 | 8805409 | 8056399 |
| 8598217 | 8423003 | 8801620 | 8053520 |
| 8597429 | 8422390 | 8801710 | 8054230 |
| 8596991 | 8422058 | 8801612 | 8054475 |
| 8599513 | 8424767 | 8803285 | 8056056 |
| 8600706 | 8426086 | 8803565 | 8057210 |
| 8598579 | 8424221 | 8802524 | 8056787 |
| 8597284 | 8423064 | 8803431 | 8057940 |
| 8598141 | 8424214 | 8805486 | 8059653 |
| 8596820 | 8422828 | 8805478 | 8059902 |
| 8597772 | 8424096 | 8806576 | 8060912 |
| 8599479 | 8426030 | 8807427 | 8061995 |
| 8600471 | 8426854 | 8809484 | 8064006 |
| 8600877 | 8427486 | 8808832 | 8062917 |
| 8602373 | 8429209 | 8809173 | 8064227 |
| 8605543 | 8426558 | 8808557 | 8063722 |
| 8604700 | 8431794 | 8806156 | 8061291 |
| 8596496 | 8423369 | 8805049 | 8060567 |
| 8600728 | 8428154 | 8806802 | 8062003 |
| 8603146 | 8430324 | 8808213 | 8063587 |
| 8601003 | 8428500 | 8808383 | 8063773 |
| 8601916 | 8429407 | 8808118 | 8064025 |
| 8601385 | 8429199 | 8807840 | 8063510 |
| 8598574 | 8426268 | 8807645 | 8062774 |
| Averages | | | |
| 8598996 | 8425027 | 8805932 | 8060228 |
| Standard Deviation | | | |
| 5037.0717 | 5725.411 | 2512.138 | 3451.861 |
| %Difference | | | |
| 0.0585774 | 0.067957 | 0.028528 | 0.042826 |

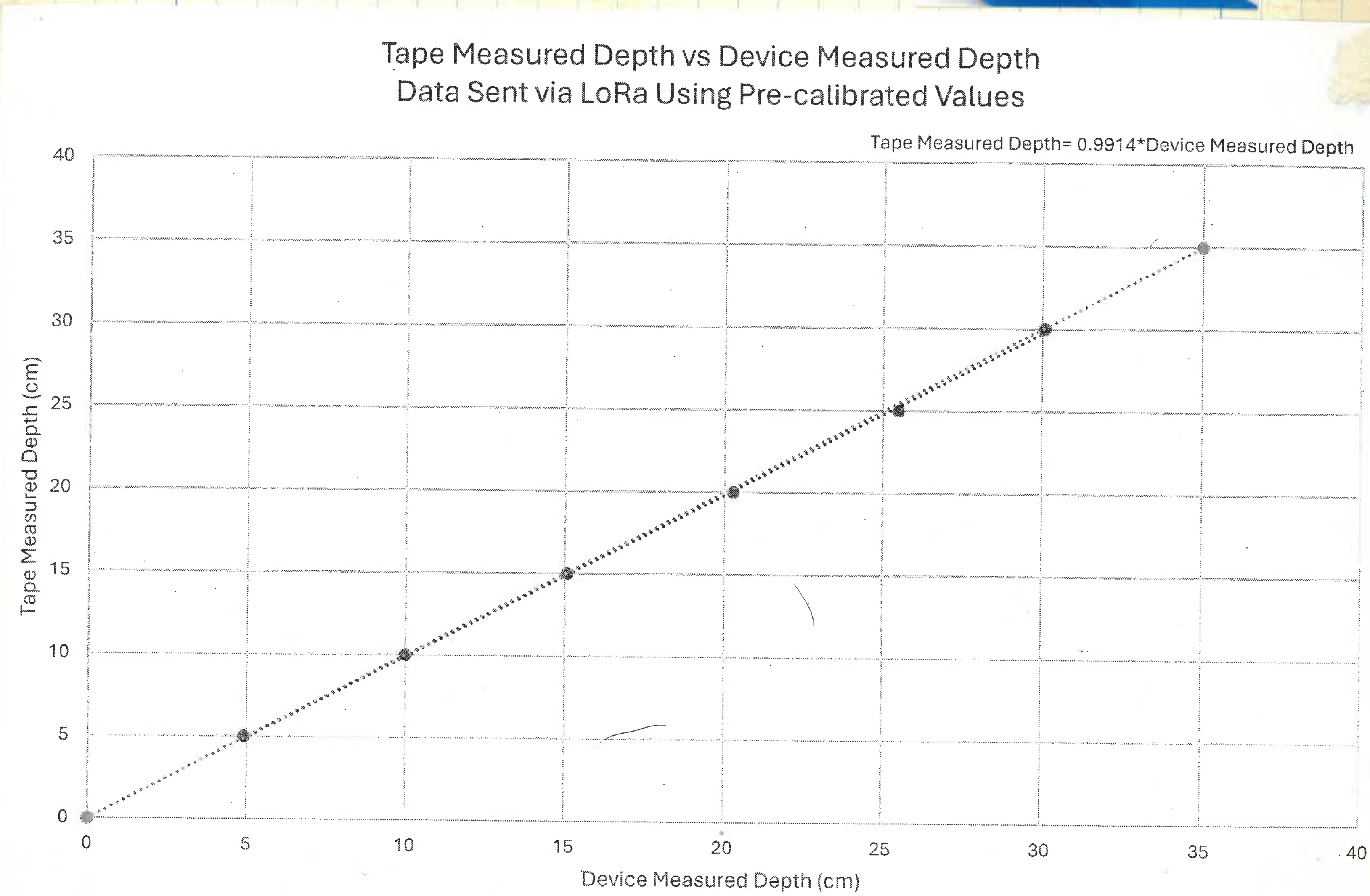
Pre-Calibrated Force Program With LoRa (For Sending Data Out)

```
#include<Q2HX711.h> //Calls Hx711 Library
#include <SPI.h>         //Calls SPI Library
#include <LoRa.h>        //Calls LoRa Library

Q2HX711 UpperHX711(2,3);  //Identifies UpperHX711 pins
Q2HX711 LowerHX711(4,5);  //Identifies Lower HX711 pins

long UpperTare = 8598996;   //Precalibrated Values (4 lines)
long LowerTare = 8425027;
long UpperWeightedTare = 8805932;
long LowerWeightedTare = 8060228;

void setup()
{
Serial.begin(9600);
Serial.println("LoRa Sender");
delay(5000);
if(!LoRa.begin(915E6))              //915 MHz
 {
 Serial.println("Starting LoRa failed");
 while(1);                  //If LoRa fails, it jams up
 }
}

void loop()
{
long UpperReading = UpperHX711.read();                    //Each Upper load cell reading
long LowerReading = LowerHX711.read();                    // Each Lower load cell reading
float MassUpper = 0.75*(UpperReading-UpperTare)/(UpperWeightedTare-UpperTare);
float MassLower = 0.75*(LowerReading-LowerTare)/(LowerWeightedTare-LowerTare);
float Force = (MassUpper + MassLower)*9.8/2;              //Force calculation
float Depth = (Force/18.5)*100.0;                         // Distance of water level rise in cm
Serial.println(Depth);                   //Print to verify packet
LoRa.beginPacket();                      //Creates packet to send
LoRa.print('H');                         //Header to Specify H2Float Data
LoRa.print(Depth);                       //Data to send in packet
LoRa.endPacket();                        //Closes packet
delay(15000);
}
```

Data sent via LoRa using Pre-calibrated Values

| Device Measured Depth (cm) | Tape Measured Depth (cm) |
|---|---|
| 4.91 | 5 |
| 9.99 | 10 |
| 15.07 | 15 |
| 20.27 | 20 |
| 25.48 | 25 |
| 30.06 | 30 |

Image of Water Level To Top Edge of Cap for PreCalibration →




<u>Left Image</u>

750g Weights Used for Calibration

<u>Right Image</u>

Tare Value Setup Used for Calibration

and Pre-Calibration Procedures To Zero Out The Load Cells. Once Complete The depth at The Top Edge of the Cap is Zero